

Universidad Técnica Estatal de Quevedo (UTEQ)

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software (Rediseño)

Asignatura: Aplicaciones Web — Período académico 2026–2027

Práctica Experimental — Unidad IV

Modelo Vista Controlador y Servicios Web: API REST, JWT y Consumo de APIs Externas

Java 21 / Spring Boot 3.5.x

Angular 17+

PostgreSQL 16

Redis Cache/Blacklist

Docker / Nginx

Integrantes (Equipo D):

Castro Espinoza Kevin Moisés
Escudero Plaza María del Rosario
Loor Medranda Marlon Taylor

Docente:

Ing. Gleiston Cicerón Guerrero Ulloa

Fecha de entrega:

Domingo, 9 de agosto de 2026

Último Commit: Lunes 10 de agosto a las 23:55

Repositorio del Proyecto (Rama: main):

Enlace al repositorio oficial en GitHub:

<https://github.com/mloorm14/practica-experimental-unidad-iv>

SOFT-A — 5to Nivel — Práctica Experimental Unidad IV | Grupal

Índice

Resumen	3
Abstract	3
1. Introducción	4
2. Desarrollo del proyecto	5
2.1. Backend	5
2.2. Frontend	5
2.3. Infraestructura	6
3. Pruebas y resultados	7
3.1. Pruebas automatizadas	7
3.2. Prueba de carga con Apache Bench	8
3.3. Auditoría de seguridad OWASP Top 10:2021	8
3.4. Verificación de los procedimientos de la guía	10
4. Tendencias tecnológicas	12
4.1. Jamstack	12
4.2. Progressive Web Apps (PWA)	12
4.3. IA generativa	12
5. Marco teórico	14
5.1. MVC y comparativa de frameworks	14
5.2. REST, JWT y OpenAPI	16
5.3. SOAP vs. REST	18
5.4. Seguridad, rendimiento y despliegue	20
6. Reflexiones individuales	22
6.1. Loor Medranda Marlon Taylor (tech lead / backend / infraestructura) . . .	22
6.2. Escudero Plaza María del Rosario (tests de integración)	23
6.3. Castro Espinoza Kevin Moisés (frontend)	23
7. Conclusiones	25
8. Trabajo futuro	25
9. Referencias	26
Anexo A: Estructura del repositorio	28

Resumen

El presente trabajo describe la práctica experimental de la Unidad IV de Aplicaciones Web, desarrollada sobre un sistema de gestión bibliotecaria compuesto por un backend Spring Boot 3.5.16 (Java 21), un frontend Angular, PostgreSQL 16, Redis y una infraestructura de despliegue basada en Docker Compose y Nginx. Durante esta unidad el equipo implementó la versión final de la API REST bajo el prefijo `/api/v1`, un formato de respuesta uniforme (`ApiResponse`), el manejo de errores estandarizado con `ProblemDetail`, la autenticación con JWT firmado en HS384 almacenado en una cookie `HttpOnly`, la autorización por roles con `@PreAuthorize`, la integración con la API externa Open Library y un servicio SOAP contract-first para la consulta de libros por ISBN. Como verificación, el repositorio cuenta con 48 pruebas automáticas (unitarias, de repositorio y 12 nuevas de integración HTTP), una prueba de carga con Apache Bench que alcanzó 986.8 requests por segundo con cache caliente y una auditoría de seguridad sobre OWASP Top 10:2021 aplicada contra el código real.

Palabras clave: API REST, JWT, Spring Boot, Testcontainers, SOAP, OpenAPI, seguridad, Docker.

Abstract

This work describes the experimental practice of Unit IV of the Web Applications course, developed on a library management system made of a Spring Boot 3.5.16 backend (Java 21), an Angular frontend, PostgreSQL 16, Redis and a deployment infrastructure based on Docker Compose and Nginx. During this unit the team implemented the final version of the REST API under the `/api/v1` prefix, a uniform response format (`ApiResponse`), standardized error handling with `ProblemDetail`, JWT authentication signed in HS384 stored in an `HttpOnly` cookie, role-based authorization with `@PreAuthorize`, the integration with the external Open Library API and a contract-first SOAP service to look up books by ISBN. As verification, the repository has 48 automated tests (unit, repository and 12 new HTTP integration tests), a load test with Apache Bench that reached 986.8 requests per second with a warm cache and an OWASP Top 10:2021 security audit applied against the real code.

Keywords: REST API, JWT, Spring Boot, Testcontainers, SOAP, OpenAPI, security, Docker.

1. Introducción

La práctica experimental de la Unidad IV continúa el sistema de gestión bibliotecaria iniciado en la Unidad III. El objetivo general del curso es que el equipo construya una aplicación web completa con arquitectura MVC, que exponga e integre servicios web REST y SOAP, y que documente su funcionamiento mediante diagramas de arquitectura y pruebas verificables.

En esta unidad el trabajo se centró en consolidar el backend y cerrar los objetivos específicos pendientes. El backend quedó versionado bajo `/api/v1`, con un envelope de respuesta uniforme para las respuestas exitosas (`ApiResponse`) y `ProblemDetail` (RFC 7807) para los errores. Se completaron cinco recursos CRUD (Libro, Autor, Editorial, Idioma y EstadoLibro), la autenticación JWT en cookie `HttpOnly` con revocación de tokens mediante una blacklist de `jti` en Redis, la autorización por rol con `@PreAuthorize`, el consumo de la API externa Open Library con `cache-aside` y un servicio SOAP `contract-first` para consultar un libro por ISBN. El frontend Angular quedó conectado a la API, con login, listado paginado y CRUD de libros, y la aplicación se despliega completa con Docker Compose (cinco servicios) detrás de un reverse proxy Nginx.

Este documento presenta el desarrollo realizado, las pruebas y resultados obtenidos, el marco teórico que sustenta las decisiones técnicas y la evidencia recopilada (prueba de carga, auditoría de seguridad y pruebas de integración). Al final se incluyen las reflexiones individuales del equipo, las conclusiones y el trabajo futuro.

2. Desarrollo del proyecto

El repositorio (<https://github.com/mloorm14/practica-experimental-unidad-iv>) contiene todo el trabajo de las tres unidades. La Unidad IV se desarrolló sobre la rama `main`, con commits convencionales en español (formato `feat:/fix:/docs:`), siguiendo la guía interna del equipo que exige el `./mvnw -B clean verify` en verde antes de cada commit del backend.

2.1. Backend

El backend es una aplicación Spring Boot 3.5.16 sobre Java 21 ([Spring Boot, s.f.](#)), con las siguientes características implementadas y verificadas en el repositorio:

- **API versionada:** todos los endpoints viven bajo `/api/v1`. (commit `e46b0b7`).
- **Envelope uniforme:** las respuestas exitosas usan `ApiResponse<T>` con los campos `success`, `data`, `message`, `errors` y `meta`. Los errores usan `ProblemDetail`, gestionados de forma centralizada en `GlobalExceptionHandler`.
- **Autenticación JWT:** el token se firma con HMAC-SHA384 (`jjwt`), viaja en una cookie `HttpOnly` con `SameSite=Strict` y también se acepta por header `Authorization: Bearer` (filtro `JwtAuthenticationFilter`).
- **Revocación de sesión:** al hacer logout, el `jti` del token se guarda en una blacklist de Redis con TTL igual a la vida restante del token (`TokenBlacklistService`).
- **Autenticación por rol:** las operaciones de escritura de los cinco recursos usan `@PreAuthorize("hasRole('ADMIN')")`, con `@EnableMethodSecurity` ([Spring Security, s.f.](#)).
- **Cinco recursos CRUD:** Libro (con relación N:M hacia Autor), Autor, Editorial, Idioma y EstadoLibro. La gestión del esquema se hace solo con migraciones Flyway versionadas (V1 a V8) y `hibernate.ddl-auto: validate`.
- **Integración externa:** `OpenLibraryClient` consulta la API de Open Library por ISBN, con cache-aside de 24 horas en un namespace Redis separado y un endpoint `GET /api/v1/libros/{id}/enriquecido` que nunca falla si el servicio externo no responde.
- **Servicio SOAP:** servicio contract-first con Spring-WS (`libro-catalogo.xsd` → JAXB), expuesto en `/ws/libro-catalogo.wsdl`, con manejo de SOAP Fault para ISBN inexistente.
- **Documentación:** OpenAPI/Swagger UI en `/api/documentation` (con alias `/api/docs`), configurado con `springdoc-openapi 2.9.0`.

2.2. Frontend

El frontend es una aplicación Angular 17+ conectada a la API mediante el servicio `libro.service.ts`, que usa `Observable<ApiResponse<T>>` y desenvuelve el `.data` del

envelope antes de llegar a los componentes. Incluye login, listado paginado de libros con ruta protegida, formulario para crear y editar libros, eliminación desde el listado y un interceptor de errores 401 con guard de autenticación.

En esta unidad Kevin amplió el frontend con cuatro entregas verificables en el historial de la rama `feature/frontend-kevin` (21 commits, mergeados a `main` en el commit 546380e):

1. **CRUD completo de Autor** (`autor.service.ts`, commit cb8132b), replicando literalmente el patrón de `libro.service.ts`: cada método tipado como `Observable<ApiResponse<` y desenvuelto con `.pipe(map(r => r.data))`, con las páginas de listado y formulario (commits 8aac2fe y 0060bab).
2. **Página de detalle de libro con Open Library** (`libro-detalle/`, commit ade913d), que consume `GET /api/v1/libros/{id}/enriquecido` y muestra `tituloOpenLibrary`, `coverUrl`, `numeroPaginas` y `descripcionOpenLibrary` con manejo explícito de campos `null` (commits 64772a4 y 42fd57f para el modelo y el método del servicio).
3. **Vistas condicionadas por rol**: el helper `esAdmin()` del `AuthService` (commit 5173124) oculta crear/editar/eliminar para el rol `USER` en libros y autores (commit e3af62c).
4. **Progressive Web App**: service worker de Angular (`@angular/service-worker`) y manifest instalable (commits 82779c9, 1532b0b y 0050774).

El detalle técnico de los tropiezos reales durante esta implementación (un error de compilación TS1205 al reutilizar un tipo entre servicios, una suposición incorrecta sobre el código de respuesta del logout) está documentado en la reflexión individual de Kevin (sección 6.3).

2.3. Infraestructura

El despliegue se definió en `docker-compose.yml` con cinco servicios: `postgres` (PostgreSQL 16; [PostgreSQL Global Development Group, s.f.](#)), `redis` (Redis 7), `app` (backend Spring Boot), `frontend` (Angular servido por Nginx) y `nginx` (reverse proxy de entrada, único puerto publicado al host). El `.env` define `JWT_SECRET` y `COOKIE_SECURE` (seguro por defecto; solo se relaja en desarrollo local). Todos los contenedores levantan con un solo comando `docker compose up -d -build`.

3. Pruebas y resultados

3.1. Pruebas automatizadas

El proyecto pasó de 36 pruebas (unitarias y de repositorio) a 48, agregando en esta unidad el paquete `backend/src/test/java/ec/edu/uteq/pfcbackend/integration/`, con 12 pruebas de integración HTTP end-to-end sobre MockMvc + Testcontainers (PostgreSQL 16 y Redis 7 reales). Las pruebas se organizaron en dos clases:

- **AuthIntegracionTest** (5 pruebas): login con credenciales válidas devuelve 200 y fija la cookie `access_token` con `HttpOnly`; login con credenciales inválidas devuelve 401; login con usuario inexistente devuelve 401; listar libros sin token devuelve 401; y logout invalida el token (el mismo token pasa de 200 a 401 tras el logout).
- **LibroIntegracionTest** (7 pruebas): listar libros con rol USER devuelve 200; crear un libro con rol USER devuelve 403; crear un libro con rol ADMIN devuelve 201; obtener un libro inexistente devuelve 404 con `ProblemDetail`; crear un libro con body inválido devuelve 400 con el arreglo `errors` poblado; eliminar un libro con rol USER devuelve 403; y una respuesta exitosa conserva el envelope `ApiResponse`.

El resultado del build completo es reproducible con el comando `./mvnw -B clean verify`: 48 pruebas ejecutadas, 0 fallos y 0 errores, con compilación, empaquetado y cobertura JaCoCo incluidos.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 48, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ pfc-backend ---
[INFO] Loading execution data file C:\Users\mescu\Documents\practica-experimental-unidad-iv\backend\target\jacoco.exec
[INFO] Analyzed bundle 'pfc-backend' with 58 classes
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ pfc-backend ---
[INFO] Building jar: C:\Users\mescu\Documents\practica-experimental-unidad-iv\backend\target\pfc-backend-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:3.5.16:repackage (repackage) @ pfc-backend ---
[INFO] Replacing main artifact C:\Users\mescu\Documents\practica-experimental-unidad-iv\backend\target\pfc-backend-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\mescu\Documents\practica-experimental-unidad-iv\backend\target\pfc-backend-0.0.1-SNAPSHOT.jar.original
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 01:09 min
```

Figura 1: Resultado del build `clean verify`: 48 pruebas ejecutadas, 0 fallos, BUILD SUCCESS.

pfc-backend

Sessions

pfc-backend

Element	Missed Instructions	Cov	Missed Branches	Cov	Missed	Cnty	Missed	Lines	Missed	Methods	Missed	Classes
ec.edu.uteq.pfcbackend.service	52%		31%		46	70	75	188	39	62	3	5
ec.edu.uteq.pfcbackend.controller	42%		60%		29	38	53	91	25	33	4	7
ec.edu.uteq.pfcbackend.ws.gsn	0%		n/a		27	27	38	38	27	27	4	4
ec.edu.uteq.pfcbackend.dto	76%		66%		11	27	9	31	7	21	6	16
ec.edu.uteq.pfcbackend.ws	0%		n/a		4	4	19	19	4	4	2	2
ec.edu.uteq.pfcbackend.entity	46%		n/a		8	12	20	36	8	12	2	6
ec.edu.uteq.pfcbackend.client	71%		25%		2	7	8	27	0	5	0	4
ec.edu.uteq.pfcbackend.oxcedion	76%		n/a		2	12	5	25	2	12	0	4
ec.edu.uteq.pfcbackend.security	92%		59%		8	22	4	55	0	11	0	3
ec.edu.uteq.pfcbackend.config	97%		50%		3	26	2	72	2	25	0	6
ec.edu.uteq.pfcbackend	37%		n/a		1	2	2	3	1	2	0	1
Total	1,028 of 2,583	60%	32 of 66	51%	141	247	235	585	115	214	21	58

Created with JaCoCo 0.8.12.202403310830

Figura 2: Reporte de cobertura JaCoCo generado por el build.

seis están completamente mitigados, uno tiene mitigación parcial (A07, con una nota concreta sobre fuerza bruta) y uno es un gap real no mitigado (rate limiting, que también afecta a A07). El resumen de la auditoría es el siguiente:

Tabla 2: Resumen de la auditoría OWASP Top 10:2021 aplicada al proyecto (vector, contramedida y evidencia de cada control).

Vulnerabilidad	Vector de ataque	de	Contramedida implementada	Evidencia
A01 Broken Access Control	Usuario USER intenta un write		@PreAuthorize("hasRole('ADMIN')") + @EnableMethodSecurity	403
A02 Cryptographic Failures	Dump de BD / robo de token		BCrypt (costo 10), JWT HS384, cookie Secure por entorno	hash \$2a\$10\$...
A03 Injection	SQL titulo	en	Repositorio parametrizado (Spring Data JPA)	curl → 0 resultados
A04 Insecure Design	Fuerza bruta / DoS de aplicación		Gap: sin rate limiting (Bucket4j disponible)	5 intentos → 5×401
A05 Security Misconfiguration	Acceso a /actuator/health		Nginx no exposición health,info	index.html / 500 genérico
A06 Vulnerable Components	CVE en librerías		Gap parcial: jjwt 0.12.6 → 0.13.0	versions:display-dependency-updates
A07 Auth Failures	Reuso de JWT robado post-logout		Blacklist de jti en Redis + TTL	misma cookie 200 → 401
XSS	onerror titulo	en	Sanitización Angular + cookie HttpOnly	0 usos de innerHTML

Entre los hallazgos relevantes: los intentos de inyección SQL sobre el parámetro **titulo** se tratan como texto literal (0 resultados, la tabla queda intacta); el login con usuario **USER** contra una operación de escritura devuelve 403; y la misma cookie reutilizada después del logout pasa de 200 a 401, verificando la blacklist de **jti** en Redis.

3.4. Verificación de los procedimientos de la guía

La guía pide verificar, en orden, una lista de procedimientos. El estado real de cada uno es el siguiente:

- Paso 1. Pruebas automatizadas (10+ pasando, 0 fallos):** el build `./mvnw -B clean verify` ejecuta 48 pruebas con 0 fallos y 0 errores, verificable en las capturas 1 a 4 de la sección 3.1.
- Paso 2. Documentación Swagger y colección Postman:** Swagger UI es accesible en `http://localhost/api/documentation`, con todos los endpoints de los cinco recursos CRUD documentados con sus esquemas. La colección de Postman exportada en `docs/postman/SGB-API.postman_collection.json` contiene **29 requests** organizadas en **6 carpetas** (Auth: 2; Libros: 7; Autores: 5; Editoriales: 5; Idiomas: 5; Estados: 5), superando el mínimo de 20 que exige la guía.
- Paso 3. API externa visible en la interfaz:** el endpoint `GET /api/v1/libros/{id}/enriquecido` enriquece el libro con los datos de Open Library. La respuesta de la API externa se almacena en Redis en el namespace `openlibrary_isbn` con TTL de 24h (`RedisCacheConfig.java`); se verifica con `docker compose exec redis redis-cli KEYS ".openlibrary_isbn*"`, que lista las claves cacheadas de cada ISBN consultado, y el cliente distingue `NoEncontrado` de `ServicioNoDisponible` para ofrecer un mensaje amigable en la interfaz cuando el proveedor externo no responde. La visualización se implementa en `frontend/src/app/pages/libro-detalle/libro-detalle.html` (componente `LibroDetalle`), que muestra `tituloOpenLibrary`, `coverUrl`, `numeroPaginas` y `descripcionOpenLibrary` con `?? 'No disponible'` como valor de respaldo en cada campo, y un placeholder ("Portada no disponible.") cuando `coverUrl` es `null` — de modo que la página nunca se rompe si Open Library no responde o el ISBN no existe ahí. Verificado en vivo para ambos casos: el libro con ISBN de "1984" (con datos) y *Cien años de soledad* (sin datos), ninguno de los dos produjo errores de JavaScript en el navegador.
- Paso 4. Headers HTTP, carga y despliegue:** la prueba de carga (`ab -n 500 -c 20`) está en la sección 3.2; el despliegue se verifica con `docker compose up -d` (sin errores) y `docker compose ps`, que muestra los cinco servicios Up con sus healthchecks. La verificación de los headers de seguridad se ejecutó en vivo contra el sistema real (`curl -I` sobre `GET /api/v1/libros` con un token JWT válido, a través de Nginx):

```
HTTP/1.1 200
Server: nginx/1.31.3
Date: Tue, 11 Aug 2026 03:29:46 GMT
Content-Type: application/json
Connection: keep-alive
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
```

Listing 1: Headers HTTP reales devueltos por el sistema (`curl -I`, 11 de agosto de 2026).

Confirma lo esperado: Spring Security no deshabilita el bloque de headers por defecto (`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Cache-Control` sin cache para respuestas autenticadas).

4. Tendencias tecnológicas

4.1. Jamstack

Jamstack ([Netlify, s.f.](#)) describe una arquitectura donde el contenido se sirve estático desde una CDN y la lógica dinámica se consume mediante APIs. Los generadores de sitios estáticos que materializan esta arquitectura se clasifican por el momento en que generan el HTML: *build-time* (Next.js estático, Nuxt, Astro) o *runtime* (Next.js con SSR o ISR), una decisión que afecta directamente a la latencia de la primera carga y al *time-to-interactive* de la aplicación. Este proyecto no adopta Jamstack completo, pero comparte una de sus ideas: el frontend Angular se construye y se sirve como estática a través de Nginx, y toda la dinámica se obtiene de la API REST. La diferencia es que aquí el reverse proxy y la API conviven en el mismo origen (<http://localhost>), lo que evita configuraciones de CORS. El equipo considera Jamstack apropiado para aplicaciones con poco estado y contenido predecible; para un catálogo con escrituras frecuentes y roles, el costo de desplegar el frontend y el backend por separado no aporta una ventaja clara.

4.2. Progressive Web Apps (PWA)

Una PWA combina una aplicación web con instalación, funcionamiento offline parcial y notificaciones ([Google, s.f.](#)). La guía del SGA pide evaluar esta tendencia y el equipo la implementó en el frontend: el service worker de Angular (`@angular/service-worker`, configurado en `ngsw-config.json`) y un manifest instalable (`manifest.webmanifest`) se verificaron en vivo con el stack completo dockerizado — el service worker se registra con estado `activated`, sirviendo `ngsw-worker.js` a través del mismo reverse proxy Nginx que expone la API. La auditoría Lighthouse sobre ese mismo stack (`docs/informe/lighthouse/`, 10 de agosto de 2026) obtuvo **Performance 94**, **Accessibility 96**, **Best Practices 100** y **SEO 82**. El equipo valora la PWA como una mejora razonable para el frontend actual: el listado de libros puede funcionar offline con un `service worker` y la instalación mejora la experiencia en móvil, sin cambiar la arquitectura del backend.

4.3. IA generativa

La IA generativa es la tendencia que más debate genera dentro del curso. El equipo la usa con una posición crítica: es útil para acelerar tareas concretas (generar estructuras de prueba, resumir documentación, explorar alternativas de configuración), pero no reemplaza la verificación. En este proyecto toda decisión de arquitectura se sostiene con evidencia real del repositorio (ADR, mediciones de Apache Bench, auditoría OWASP) y todo código generado con apoyo de herramientas se revisa, se ejecuta y se cubre con pruebas antes de considerarse terminado. El riesgo que el equipo identifica en la adopción irreflexiva de esta tecnología es que produce texto y código aparentemente correctos que un humano sin criterio técnico puede incorporar sin verificar, y eso se nota en informes con referencias inventadas o en código que no compila. La postura del equipo es usarla como apoyo y mantener la responsabilidad de la revisión en las personas. En la práctica la herramienta se usó en dos modalidades distintas: los asistentes de autocompletado (Copilot) para código en contexto dentro del IDE, y el *scaffolding* de plantillas (por ejemplo, la generación inicial de un controlador CRUD). Ambas modalidades exponen el mismo límite ético: los modelos

no garantizan originalidad ni trazabilidad de su output, por lo que el equipo verificó el código generado, lo firmó con su autoría y cubrió cada fragmento con pruebas; no se incorporó ningún texto ni código sin revisión humana. La responsabilidad final de un error introducido por una herramienta sigue siendo del equipo, y las decisiones de diseño se argumentaron con mediciones propias en lugar de citar afirmaciones no verificables.

5. Marco teórico

5.1. MVC y comparativa de frameworks

El patrón Modelo-Vista-Controlador fue descrito originalmente por Trygve Reenskaug en 1979 dentro del entorno Smalltalk-80: el *modelo* gestiona el estado de la aplicación, la *vista* es la representación de ese estado y el *controlador* es el manejador de las entradas del usuario. El modelo web adaptó esta separación: el controlador ya no recibe eventos de teclado o ratón sino solicitudes HTTP; la vista se renderiza en el servidor con un motor de plantillas o en el cliente (SPA); y el modelo pasa a incluir repositorios y servicios, no solo el estado en memoria. De esa adaptación surgieron variantes como MVP (*Model-View-Presenter*), donde la vista no tiene referencia directa al modelo, y MVVM (*Model-View-ViewModel*), que usa un enlace de datos bidireccional y es el patrón de facto en SPA modernas (Fowler, 2002) [2]. El backend de este proyecto aplica MVC en su forma en capas (controlador, servicio, repositorio), tal como documenta el ADR-001 de la Unidad III, y las entidades mapeadas con JPA/Hibernate actúan como el modelo.

Todos los frameworks MVC web usan el patrón *Front Controller*: un punto de entrada único (en PHP, `index.php`; en .NET, `Program.cs`; en Spring Boot, `DispatcherServlet`). Centralizar la entrada permite manejar errores, autenticación, logging y enrutamiento en un solo lugar, en vez de repetirlos en cada página. En este proyecto el ciclo completo de una petición es el siguiente: (1) la solicitud entra por Nginx, que enruta `/api/` hacia el backend; (2) `DispatcherServlet` selecciona el controlador a partir de la anotación `@RequestMapping`; (3) la cadena de filtros de Spring Security (`JwtAuthenticationFilter`) extrae el JWT de la cookie o del header `Authorization`, valida la firma y consulta la blacklist en Redis, estableciendo el contexto de seguridad; (4) el controlador recibe el request y valida el body con `@Valid`; (5) el servicio aplica la lógica de negocio y el repositorio de Spring Data JPA consulta PostgreSQL, con el cache-aside en Redis interviniendo en las lecturas marcadas con `@Cacheable`; (6) la respuesta se serializa en el envelope uniforme `ApiResponse` y se devuelve con el código HTTP correspondiente. Los pasos 4 a 6 se materializan en el código real siguiente (el controlador, la capa de servicio con cache y el envelope):

```
@RestController
@RequestMapping("/api/v1/libros")
public class LibroController {
    @GetMapping
    @Operation(summary = "Listar libros, con filtro opcional por titulo")
    public ApiResponse<List<LibroResponse>> listar(
        @RequestParam(required = false) String titulo, Pageable pageable) {
        Page<LibroResponse> pagina = libroService.listar(titulo, pageable);
        return new ApiResponse<>(pagina.getContent(), pagina);
    }
}

@Service
public class LibroServiceImpl {
    @Cacheable(value = CACHE_LISTADO,
        key = "#titulo + '-' + #pageable.pageNumber + '-' + #pageable.pageSize +
        ↪ ...")
    public Page<LibroResponse> listar(String titulo, Pageable pageable) {
        return (titulo == null || titulo.isBlank())
```

```

        ? libroRepository.findAll(pageable)
        : libroRepository.findByTituloContainingIgnoreCase(titulo, pageable);
    }
}

```

Listing 2: Controlador, servicio y cache del listado (fragmento real de `LibroController.java` y `LibroServiceImpl.java`).

Sobre la elección del framework, el ADR-005 compara las tres opciones que el propio PDF de la guía menciona como válidas para este curso. La tabla siguiente extiende esa comparación a los cinco frameworks que la guía exige considerar (Laravel, Symfony, Spring Boot, ASP.NET Core y Django), con los criterios solicitados; la fila de rendimiento usa la misma fuente que el ADR-005 (TechEmpower, Round 23, prueba Fortunes), por ser la ronda publicada más reciente antes del cierre del proyecto, y el resto de los criterios se tomaron de la documentación oficial de cada framework ([Otwell, 2024](#)) [1]:

Tabla 3: Comparativa de frameworks MVC (criterios de la guía, datos de ADR-005 y documentación oficial).

Criterio	Laravel 11.x	Symfony 7.x	Spring Boot 3.x	ASP.NET Core 8.x	Django 5.x
Paradigma	Convención sobre con-fig.	Explícito	Explícito	Intermedio	Convención sobre con-fig.
ORM incluido	Eloquent	Doctrine	Spring Data JPA	EF Core	Django ORM
Motor de plantillas	Blade	Twig	Thymeleaf	Razor	Django Templates
Ecosistema	Paquetes (Packa-gist)	Paquetes	Starters (Maven)	NuGet	Apps (Py-PI)
Rendimiento (TechEmpower R23, Fortunes)	16 800 req/s	no medi-do en el ADR	243 639 req/s	609 966 req/s	no medi-do en el ADR
Adopción LATAM	Alta (agen-cias/freelance)	Media	Alta (empresarial/banca)	Media	Media

El equipo eligió Spring Boot principalmente por tres razones documentadas: la experiencia previa real con Java, el tipado estático (que detecta errores en tiempo de compilación, como el desajuste de tipos detectado en el ADR-004) y un ecosistema de testing maduro (Testcontainers levantando un PostgreSQL real en las pruebas, sin mocks de base de datos). Se reconoce en el propio ADR-005 la contraparte honesta: Java es más verboso que Laravel para el mismo CRUD (cada recurso requirió siete archivos), y la configuración inicial de Spring es más pesada que la de un microframework. La elección se justifica además por el rendimiento medido en este mismo proyecto (P99 de 30 a 36 ms en la prueba de carga), que confirma que el rendimiento bruto de ASP.NET Core no era la restricción activa del PFC (ADR-005).

5.2. REST, JWT y OpenAPI

HTTP es un protocolo sin estado por diseño: cada petición es independiente y el servidor no guarda memoria de interacciones previas (Fielding, 2000; Fielding et al., 2022) [3]. REST se apoya en seis principios que la API de este proyecto aplica (o relega, cuando no aplica) de forma explícita y verificable:

- **Cliente-Servidor:** separación total entre el frontend Angular y la API; el backend nunca genera HTML y el frontend nunca accede a la base de datos.
- **Stateless:** el servidor no guarda sesión; la identidad viaja en el JWT de cada petición. Esto es lo que habilita escalar horizontalmente sin *sticky sessions*.
- **Cacheable:** cada recurso declara con **Cache-Control** si su respuesta puede almacenarse; el cache-aside en Redis (un TTL por dominio) es la implementación concreta de este principio.
- **Interface uniforme:** identificación de recursos por URI, representación JSON auto-descriptiva (**ApiResponse**), mensajes autodescriptivos y HATEOAS como nivel de madurez que el proyecto alcanza parcialmente (enlaces en el envelope) sin hipermedia completa.
- **Layered System:** frontend, Nginx, backend, Redis y PostgreSQL forman una pila en la que cada capa solo conoce a su vecina; Nginx esconde la topología interna al cliente.
- **Code-on-Demand:** opcional por definición; no se usa, porque introducir código descargado complica la seguridad sin aportar al PFC.

La jerarquía de la URI es el contrato de esa interface uniforme, y su diseño sigue las reglas de la arquitectura REST (Richardson & Ruby, 2007) [4]: sustantivos en plural (`/libros`, no `/getLibros`), anidamiento solo para subrecursos legítimos, sin verbos en la URL y con el versionado en el prefijo `/api/v1`. El diseño completo de la API es:

- `POST /api/v1/auth/login` y `POST /api/v1/auth/logout`
- `GET /api/v1/libros` (con filtros), `GET /api/v1/libros/{id}`, `POST /api/v1/libros`, `PUT /api/v1/libros/{id}`, `DELETE /api/v1/libros/{id}` y `GET /api/v1/libros/{id}/enriquecer` (consume la API externa)
- `GET|POST|PUT|DELETE /api/v1/autores/{id}`, lo mismo para `/editoriales`, `/idiomas` y `/estados`
- `GET /actuator/health` para el healthcheck de Docker

La estructura del JWT es la de la especificación RFC 7519 (Jones et al., 2015): un **Header** con el algoritmo (HMAC-SHA384 en este proyecto), un **Payload** con los claims `sub` (id de usuario), `username`, `roles`, `iat` y `exp`, y una **Signature** calculada con el secreto de 48 bytes. El flujo es: `login` valida credenciales contra PostgreSQL → el servidor firma el JWT y lo entrega en una cookie `HttpOnly` con `SameSite=Strict` (también aceptado por `Authorization: Bearer`) → en cada request autenticada, `JwtAuthenticationFilter`

valida la firma y la expiración y establece el `SecurityContext` → en el `logout`, el `jti` entra en una blacklist de Redis ([Redis, s.f.](#)) con TTL. La cookie `HttpOnly` no es legible desde JavaScript, lo que elimina el vector de robo por XSS que existiría con `localStorage`; y la blacklist resuelve el problema estructural del JWT de que un token firmado no puede revocarse (la estrategia de corto TTL + revocación explícita se documenta en la guía como la vía correcta).

OpenAPI ([OpenAPI Initiative, 2023](#)) actúa como el contrato entre frontend y backend: la especificación se genera con `springdoc-openapi` a partir de las anotaciones de los controladores y se sirve en `/api/documentation`. Para cada endpoint la especificación declara `operationId`, parámetros de ruta y query, el `requestBody` con su `reference` al esquema y las respuestas por código con sus esquemas, de modo que Angular puede generar su cliente tipado desde el mismo documento. Para visualizar ese contrato se compararon dos alternativas: **Swagger UI**, interactivo (permite ejecutar las peticiones desde el navegador), y **Redoc**, estático y pensado como documentación de lectura; el proyecto usa Swagger UI por su interactividad y lo expone en `/swagger-ui.html`. La documentación queda sincronizada con el código, a diferencia de documentación escrita a mano que tiende a quedar desactualizada. El fragmento siguiente es la entrada que `springdoc-openapi` genera para el login (resumida), y muestra los elementos que la guía pide identificar: `operationId`, parámetros, `requestBody` con su esquema y las respuestas por código:

```
paths:
  /api/v1/auth/login:
    post:
      operationId: login
      summary: Autentica un usuario y devuelve un JWT
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/LoginRequest' # username, password
      responses:
        '200':
          description: Login exitoso; el JWT viaja en la cookie access_token
          headers:
            Set-Cookie:
              schema: { type: string }
        '401':
          description: Credenciales invalidas
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ProblemDetail'
  components:
    securitySchemes:
      bearerAuth:
        type: http
        scheme: bearer
        bearerFormat: JWT
```

Listing 3: Documentación OpenAPI del endpoint de login (generada por `springdoc-openapi`, resumida).

5.3. SOAP vs. REST

SOAP es un protocolo de mensajería basado en XML con un contrato formal (WSDL) ([World Wide Web Consortium, 2000](#)). Estructuralmente, un mensaje SOAP se compone de un **Envelope** que envuelve al **Header** (metadatos, seguridades WS-Security) y al **Body** (la operación), y los errores viajan como **SOAP Fault**; cada request indica la operación a ejecutar con el header **SOAPAction**, y el transporte no se limita a HTTP, sino que puede ser SMTP o TCP. REST, en cambio, es un estilo arquitectónico sobre HTTP en el que los recursos se manipulan con los verbos del protocolo ([Fielding, 2000](#)) [3]. La comparación entre frameworks y protocolos que aquí se resume sigue la taxonomía de referencia del SWEBOK en su edición 4.0 ([Washizaki et al., 2024](#)) [5], que estructura el cuerpo de conocimiento en ingeniería de software; el proyecto implementa ambos protocolos, lo que permite compararlos con evidencia real.

Tabla 4: Comparación SOAP vs. REST aplicada a los servicios implementados.

Criterio	SOAP (implementado)	REST (implementado)
Formato de mensaje	XML (SOAP Envelope)	JSON
Tipado	Estricto (XSD, contract-first)	Definido por el esquema OpenAPI
Overhead de serialización	Alto (namespaces)	Bajo
Contrato	WSDL (contract-first)	OpenAPI (descripción)
Descubrimiento	WSDL auto-generado	Swagger UI
Transporte	HTTP (también SMTP/JMS)	HTTP(S)
Manejo de errores	SOAP Fault (HTTP 500)	Códigos de estado + ProblemDetail
Cacheabilidad	Difícil (POST)	Explícita (GET cacheable)
Tamaño de payload (medido)	564 bytes / 8 campos	550 bytes / 16 campos
Seguridad	WS-Security	HTTPS + JWT
Complejidad de implementación	Alta	Baja-media
Interoperabilidad	Alta (sistemas corporativos)	Amplia en la web
Idempotencia	No definida por SOAP	GET/PUT/DELETE por diseño
Facilidad desde JavaScript	Baja (XML + WSDL)	Alta (JSON nativo)
Casos de uso actuales (2025)	Banca, sector público, SRI	APIs web, móvil, SPA

La medición real de payload del mismo caso de uso (consultar el libro “1984”) arroja un resultado interesante: la respuesta REST es 14 bytes más pequeña que la SOAP a pesar de traer el doble de campos, porque cada elemento XML necesita etiquetas de apertura y cierre con namespace. La respuesta SOAP para un ISBN inexistente devuelve un **SOAP Fault** bien formado con el mensaje exacto de la causa, mientras el equivalente REST devuelve un **ProblemDetail** con su código de estado. El equipo mantiene el servicio SOAP público a propósito, como demostración del resultado de aprendizaje que exige exponer ambos tipos de servicio, y no como recurso de negocio que necesite protección por rol.

El SOAP no es una tecnología muerta en 2025: sigue siendo el estándar donde la interoperabilidad entre instituciones grandes importa más que la simplicidad. Los casos de uso vigentes se concentran en la banca (servicios de core bancario entre instituciones, compensación y clearing), en el sector público ecuatoriano (intercambio de datos entre

instituciones del Estado a través de sus *service bus*) y en el SRI, que publica servicios SOAP/WSDL para la validación de comprobantes electrónicos; son sistemas donde los clientes están controlados (no se consumen desde el navegador) y donde el contrato WSDL firmado reduce ambigüedad contractual. Ninguno de esos casos justifica hoy una API pública nueva en SOAP, pero un desarrollador ecuatoriano los va a encontrar en su carrera profesional, y por eso esta práctica los implementa.

Para consumir una API externa, la guía pide comparar los clientes HTTP de los tres stack que se comparan en la Unidad III: **GuzzleHTTP** (PHP), **HttpClient** (.NET) y **RestTemplate/WebClient** (Java). Los tres cubren peticiones GET/POST con headers y JSON, pero se diferencian en tres dimensiones prácticas: los **timeouts** de conexión y respuesta (configurables en los tres, en este proyecto 5 s de conexión y 10 s de respuesta vía **WebClientConfig**), el **retry con backoff** ante fallos transitorios (Guzzle y HttpClient lo traen como middleware nativo; en WebClient hay que componerlo con Reactor) y la **conurrencia**: RestTemplate bloquea el hilo mientras espera la respuesta, mientras que WebClient es reactivo y no bloquea, lo que escala mejor bajo carga concurrente. El fragmento siguiente es el consumo real de Open Library en el endpoint */enriquecido*, y muestra el manejo explícito de los tres tipos de fallo que una API externa puede producir:

```
@Cacheable(value = "openlibrary_isbn", key = "#isbn",
    unless =
        ↪ "#result.getClass().getSimpleName().equals('ServicioNoDisponible')")
public OpenLibraryResult buscarPorIsbn(String isbn) {
    String isbnNormalizado = isbn.replaceAll("[^0-9Xx]", "");
    try {
        OpenLibraryResponse datos = openLibraryWebClient.get()
            .uri("/isbn/{isbn}.json", isbnNormalizado)
            .retrieve()
            .bodyToMono(OpenLibraryResponse.class)
            .timeout(TIMEOUT_RESPUESTA) // 10 s
            .block();

        return datos != null ? new Encontrado(datos) : new NoEncontrado();
    } catch (WebClientResponseException.NotFound ex) {
        return new NoEncontrado(); // 404: dato definitivo, se cachea
    } catch (WebClientResponseException ex) {
        return new ServicioNoDisponible(); // 5xx del proveedor
    } catch (WebClientRequestException ex) {
        return new ServicioNoDisponible(); // red caída, DNS, conexión
    } catch (RuntimeException ex) {
        if (ex.getCause() instanceof TimeoutException) {
            return new ServicioNoDisponible(); // no responde a tiempo
        }
        throw ex;
    }
}
```

Listing 4: Consumo de API externa con manejo de errores (fragmento de `OpenLibraryClient.java`).

La distinción entre `NoEncontrado` y `ServicioNoDisponible` no es un detalle: el *cache-aside* con la anotación `@Cacheable` de Spring Cache ([Redis](#), s.f.) guarda por defecto la primera respuesta de cada ISBN, y si se cachea un fallo transitorio, la API quedaría congelada.^{en} estado de error durante todo el TTL. Por eso el `unless` impide cachear `ServicioNoDisponible` pero sí cachea el 404, que es un dato definitivo. Cachear respuestas

externas está justificado por tres razones: respetar el *rate limiting* del proveedor, reducir la latencia percibida (una consulta externa puede tardar cientos de milisegundos) y tolerar la indisponibilidad del proveedor. El TTL se decide por dominio: datos meteorológicos (cambian cada hora) con TTL corto de 10 minutos, tipos de cambio (1 hora) y datos prácticamente estáticos como el metadato de un ISBN (24 horas); un TTL único para todos los dominios sería incorrecto para cualquiera de ellos.

5.4. Seguridad, rendimiento y despliegue

El proyecto aplica buenas prácticas de seguridad verificadas en la auditoría OWASP ([OWASP Foundation, 2021](#)) [6]. Las contraseñas se almacenan con BCrypt (costo 10, hash `$2a$10$...`); el JWT se firma con HMAC-SHA384 (secreto de 48 bytes base64), rechazando tokens sin firma o con algoritmo distinto; la cookie de sesión es segura por defecto y condicionada por entorno (antes de esta unidad estaba fijada en `false`); y las consultas se ejecutan parametrizadas por Spring Data, sin concatenación de strings SQL. El diseño defiende la capa HTTP con los headers que Spring Security habilita por defecto (el bloque de headers no está deshabilitado en `SecurityConfig`): `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` y `Cache-Control` en las respuestas, verificados en vivo con `curl -I` en la sección 3.1 (Verificación de los procedimientos de la guía, paso 4).

Sobre rendimiento, la prueba de carga siguió la metodología de la guía (conurrencia `-c`, volumen `-n` y endpoint objetivo): el comando exacto fue `ab -n 500 -c 20 -H .^authorization: Bearer <token>"http://nginx/api/v1/libros`, ejecutado desde un contenedor descartable en la misma red de Docker para no instalar nada en el host ([Apache Software Foundation, s.f.](#)). El endpoint más usado respondió con **986.80 req/s con cache caliente** frente a **802.66 req/s con cache fría**, y todos los percentiles mejoraron: P50 de 23 a 19 ms, P95 de 32 a 27 ms, P99 de 36 a 30 ms. El **error rate fue 0 % en ambas corridas** (1000 requests totales). Estos números sustentan que, para el volumen esperado de un sistema académico, la configuración actual no presenta cuellos de botella. La interpretación de un sistema que degrada suavemente bajo carga y no produce errores en picos sigue el marco de *Release It!* ([Nygard, 2018](#)) [7], que separa la resistencia estructural (la aplicación no se cae ante fallos) del rendimiento puro.

El despliegue se define con Docker Compose ([Docker, s.f.](#)): PostgreSQL, Redis, backend, frontend y Nginx como reverse proxy. Las relaciones entre servicios son explícitas en el compose: la app depende de `postgres` y `redis` (que llegan a estar saludables vía healthchecks antes de arrancar), y `nginx` es el único punto de entrada al mundo exterior. Nginx enruta `/api/` hacia el backend y `/` hacia el frontend, y no expone `/actuator/*`, lo que impide acceder a endpoints de monitorización desde el exterior; además, el backend solo habilita `health` e `info` de Actuator. Docker aporta tres beneficios concretos a este proyecto: **entornos reproducibles** (el mismo `docker-compose.yml` levanta el sistema idéntico en cualquier máquina, sin configurar JDK, Node, PostgreSQL ni Redis a mano), **aislamiento de dependencias** (cada servicio encapsula sus librerías y versiones, evitando el clásico `it works on my machine`) y la base para **CI/CD** (los pipelines pueden construir la misma imagen que se ejecuta en producción). Los secretos y valores de configuración se inyectan con variables de entorno a través de `environment:` en el compose y `${VARIABLE}` en `application.yml` (por ejemplo `JWT_SECRET`, `COOKIE_SECURE`, `DB_*`); nunca en el código fuente, porque un archivo con secretos hardcoded se filtra si el repositorio se hace público o se comparte, y no puede cambiar entre entornos sin recompilar. El despliegue

completo se levanta con `docker compose up -d` y se verifica con `docker compose ps`, que muestra los cinco servicios Up con sus healthchecks.

6. Reflexiones individuales

6.1. Loor Medranda Marlon Taylor (tech lead / backend / infra-estructura)

A lo largo de este proyecto tuve que corregir tres errores propios que, revisados en retrospectiva, dejan lecciones más útiles que cualquier cosa que hubiera aprendido si todo hubiese salido bien a la primera.

El primero fue un login roto en producción local, causado por versionar la API a `/api/v1` sin actualizar en el mismo cambio los matchers de `SecurityConfig`, que seguían apuntando a las rutas sin versionar (commit `8e73f02`). El síntoma no apareció en ningún test unitario ni de compilación: apareció al probar el flujo real en vivo, porque `SecurityConfig` y los controladores viven en archivos distintos y nada obliga a que cambien juntos. La lección no es "tener cuidado", que es inútil como lección porque no es accionable — es que un cambio de contrato de API (una ruta, un prefijo, un nombre de campo expuesto) toca más superficie de la que el diff muestra a primera vista, y que verificar en vivo después de cada cambio estructural — no solo correr los tests — no es un paso opcional ni redundante, incluso cuando "solo" se está agregando un prefijo de versión.

El segundo fue un error de comparación `==` en vez de `.equals()` dentro de una expresión SpEL de `@Cacheable(unless=...)` en `OpenLibraryClient` (corregido en el commit `c80426b`). Esa condición determinaba si un resultado de Open Library debía cachearse o no, y en las pruebas manuales que hice antes de la corrección "funcionaba" — porque el caso que probé casualmente caía del lado correcto de la comparación, no porque la comparación fuera correcta. La lección real ahí es que pasar una prueba manual no confirma que el mecanismo subyacente sea correcto; confirma que ese caso particular no expuso el defecto. Un lenguaje/motor de expresiones que no garantiza comparación por identidad de referencia versus por valor es exactamente el tipo de detalle que un test "parece que funciona" no detecta, y que solo aparece si se piensa explícitamente en el contrato del lenguaje, no en el resultado observado una vez.

El tercero lo encontramos nosotros mismos, auditando nuestro propio código con el mismo rigor que le exigiríamos a un tercero: una cookie de sesión con `secure(false)` hardcodeado en `AuthController`, sin ningún mecanismo que lo condicionara al entorno de despliegue (corregido en el commit `aa3e6b3`, ya con `app.cookie.secure` configurable y seguro por defecto). Nadie fuera del equipo lo señaló; lo encontramos porque decidimos hacer la auditoría OWASP contra el código real en vez de tratarla como un formalismo documental para cumplir un punto del PDF. Esa es la lección que más me deja este bloque: revisar el propio trabajo con la misma rigurosidad que se le exigiría a un tercero no es un ejercicio vacío ni redundante — encuentra fallos reales que de otra forma solo aparecerían en producción, o peor, no aparecerían nunca porque nadie los busca.

Si empezara este proyecto de nuevo, escribiría desde el día uno un checklist corto de "qué verificar en vivo tras cada cambio de contrato de API" (autenticación, cada rol relevante, al menos un caso de error esperado) en vez de descubrir la necesidad de ese checklist de forma reactiva, después de que un bug de ese tipo ya llegó a producción local una vez.

6.2. Escudero Plaza María del Rosario (tests de integración)

En esta unidad mi tarea fue escribir los tests de integración del backend. La guía pedía mínimo 10 tests y dejé 12, repartidos en dos clases: una para autenticación (login, logout y acceso sin token) y otra para el CRUD de libros y los permisos por rol.

Lo que más me costó no fue escribir los tests, sino hacer que funcionaran. Mi computadora tiene Java 25 instalado y el proyecto usa Java 21, así que al principio los tests ni siquiera compilaban: Lombok no generaba el código y tuve que correr el build con una bandera especial (`-Dmaven.compiler.proc=full`). Ahí entendí que el problema no era el código, sino que mi entorno no coincidía con el del proyecto.

También asumí cosas que no eran. Por ejemplo, pensé que el logout devolvía un 200 con un cuerpo, pero en realidad devuelve 204 (sin contenido). Y al probar la creación de libros, el test fallaba porque el formulario necesita más campos de los que yo le mandaba (año de publicación, editorial, idioma, estado y stock). Tuve que leer bien el DTO y el controlador para arreglarlo. Eso me enseñó a revisar el código antes de asumir cómo funciona un endpoint.

Otro problema fue con los contenedores de prueba. Al separar los tests en dos clases, compartí los mismos contenedores de base de datos y Redis, y cuando la primera clase terminaba, la segunda ya no podía conectarse y todo devolvía 500. La solución fue darle a cada clase sus propios contenedores. Me quedó claro que los tests tienen que estar aislados entre sí.

Al final el proyecto completo pasa con 48 tests y cero fallos. Si tuviera que repetir la unidad, empezaría leyendo el código de cada endpoint antes de escribir el primer test.

6.3. Castro Espinoza Kevin Moisés (frontend)

Mi parte de esta unidad fue el frontend Angular: implementar el CRUD de Autor, integrar Open Library en la vista de detalle, condicionar la UI por rol y convertir la app en PWA. El primer tropiezo fue técnico y muy concreto: al extraer la interfaz `PageResponse` a un modelo compartido para reutilizarla entre `libro.service.ts` y el nuevo `autor.service.ts`, el build falló con TS1205 porque el proyecto usa `isolatedModules` y TypeScript exige `export type` al re-exportar tipos. Fue un recordatorio directo de que copiar el patrón de un archivo existente no basta si no se entiende la configuración del compilador que lo sostiene.

Aprender a desenvolver el `ApiResponse` con `.pipe(map(r => r.data))` en cada método, tal como ya hacía `libro.service.ts`, me dejó claro por qué el contrato del backend (envelope en éxito, `ProblemDetail` en error) se traduce en que el componente nunca ve el JSON crudo. También entendí el punto de la guía sobre los campos de Open Library que pueden venir null: el DTO `LibroEnriquecidoResponse` documenta que si el ISBN no existe en Open Library la respuesta sigue siendo 200, y la vista debía manejar ese caso con placeholders en vez de romperse.

Finalmente, el rol que viaja en el body del login (`LoginResponse(username, rol)`) se guarda en el estado del `AuthService` y condiciona crear/editar/eliminar con `*ngIf`; la cookie `HttpOnly` hace inviable decodificar el JWT en el frontend, así que confiar en el body del login era lo correcto. La PWA se agregó manualmente porque `ng add @angular/pwa` no resolvió el esquema con el builder de aplicación (clave `browser` en `angular.json`);

el reporte Lighthouse quedó en `docs/informe/lighthouse` con performance 94 y best practices 100.

7. Conclusiones

1. La API REST quedó completa y versionada bajo `/api/v1`, con un formato de respuesta uniforme y errores estandarizados. Esto hace que los clientes (frontend, SOAP, pruebas) traten la API de forma consistente.
2. La autenticación JWT en cookie `HttpOnly` con blacklist de `jti` en Redis cumple el requisito de sesión sin estado del servidor y permite revocar tokens de forma efectiva, verificado en pruebas: el mismo token pasa de 200 a 401 después del logout.
3. La autorización por rol (`@PreAuthorize`) se verifica con pruebas de integración: un usuario `USER` recibe 403 en las operaciones de escritura mientras que `ADMIN` las ejecuta con 201.
4. El proyecto expone e integra servicios web REST y SOAP. La comparación con datos reales muestra que la respuesta REST es más pequeña que la SOAP incluso trayendo el doble de campos, y que el servicio SOAP devuelve `SOAP Fault` bien formado para errores.
5. La prueba de carga y la auditoría OWASP aportan evidencia real de rendimiento y seguridad: 986.80 req/s con cache caliente sin requests fallidos, y seis de ocho controles OWASP completamente mitigados, con los gaps documentados de forma honesta (rate limiting, dependencias desactualizadas).
6. Las pruebas automatizadas pasaron de 36 a 48 con la incorporación de 12 tests de integración HTTP, y todo el build (`./mvnw -B clean verify`) termina en verde, lo que da confianza para continuar evolucionando el sistema.

8. Trabajo futuro

- Aplicar la migración V9 para eliminar el campo legado `libros.autor` (String), una vez que el frontend consuma la relación N:M real con la entidad Autor.
- Implementar rate limiting en los endpoints de autenticación (por ejemplo, con Bucket4j y Redis), documentado como gap real en la auditoría OWASP (A04/A07).
- Actualizar `jjwt` de 0.12.6 a 0.13.0 por tocar directamente la superficie de autenticación, y planificar la migración a Spring Boot 4 en un bloque dedicado.
- Mitigar el riesgo de cache stampede del ADR-003 (por ejemplo, con `@Cacheable(sync=true)` o invalidación selectiva por página).
- Agregar a la API REST un endpoint de búsqueda por ISBN equivalente al del servicio SOAP, para comparar ambos servicios sobre el mismo contrato de datos.
- Completar la PWA del frontend y registrar el reporte Lighthouse como evidencia.

9. Referencias

La guía de práctica exige referencias en norma **IEEE** y el enunciado del SGA exige norma **APA**. Siguiendo la regla del equipo de cumplir ambos requisitos cuando el SGA añade una exigencia que la guía no contradice (decisión documentada en **DECISIONES-GUIA-VS-ENUNCIADO-U4.md**), las citas en el cuerpo se presentan en autor-año (APA) y, para las fuentes que la guía exige como mínimo, también con su número de cita IEEE [n] en el mismo lugar del texto; ese número corresponde a la lista IEEE que aparece al final de esta sección. Se presentan los dos listados: el primero en APA (para el SGA) y el segundo con las referencias mínimas IEEE que exige la guía.

Referencias en norma APA (exigidas por el SGA)

- Apache Software Foundation. (s.f.). *ab — Apache HTTP server benchmarking tool*. Recuperado el 10 de agosto de 2026, de <https://httpd.apache.org/docs/current/programs/ab.html>
- Breeding, M. (2017). Chapter 2. Koha. The original open source ILS. *Library Technology Reports*, 53(6). <https://journals.ala.org/index.php/ltr/article/view/6405/8452>
- Docker. (s.f.). *Docker docs*. Recuperado el 10 de agosto de 2026, de <https://docs.docker.com/>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP Semantics* (RFC 9110). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
- Google. (s.f.). *Learn PWA*. Recuperado el 10 de agosto de 2026, de <https://web.dev/progressive-web-apps/>
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7519>
- Netlify. (s.f.). *Jamstack*. Recuperado el 10 de agosto de 2026, de <https://jamstack.org/>
- Nygard, M. (2018). *Release It! Design and deploy production-ready software* (2.^a ed.). Pragmatic Bookshelf.
- OpenAPI Initiative. (2023). *OpenAPI Specification 3.1*. <https://spec.openapis.org/oas/latest.html>
- Otwell, T. (2024). *Laravel 11.x documentation*. Recuperado el 10 de agosto de 2026, de <https://laravel.com/docs/11.x>
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>

- PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation*. Recuperado el 10 de agosto de 2026, de <https://www.postgresql.org/docs/>
- Redis. (s.f.). *Redis documentation*. Recuperado el 10 de agosto de 2026, de <https://redis.io/docs/latest/>
- Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.
- Spring Boot. (s.f.). *Spring Boot reference documentation*. Recuperado el 10 de agosto de 2026, de <https://docs.spring.io/spring-boot/>
- Spring Security. (s.f.). *Spring Security reference*. Recuperado el 10 de agosto de 2026, de <https://docs.spring.io/spring-security/>
- Washizaki, H. (Ed.). (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK), Version 4.0*. IEEE Computer Society. <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- World Wide Web Consortium. (2000). *SOAP Version 1.1* (W3C Note). <https://www.w3.org/TR/soap/>

Referencias mínimas en norma IEEE (exigidas por la guía de práctica)

- [1] T. Otwell, “Laravel 11.x Documentation,” Laravel LLC, 2024. [Online]. Available: <https://laravel.com/docs/11.x>
- [2] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002, ISBN: 978-0-321-12521-7.
- [3] R. T. Fielding, “Architectural Styles and the Design of Network-Based Software Architectures,” Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000. [Online]. Available: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [4] L. Richardson and S. Ruby, *RESTful Web Services*. Sebastopol, CA, USA: O'Reilly Media, 2007, ISBN: 978-0-596-52926-0.
- [5] H. Washizaki, Ed., *Guide to the Software Engineering Body of Knowledge (SWE-BOK), Version 4.0*. IEEE Computer Society, 2024.
- [6] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: <https://owasp.org/Top10/>
- [7] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, 2018, ISBN: 978-1-68050-239-8.

Anexo A: Estructura del repositorio (versión final en main)

La estructura final del repositorio en main, generada a partir del árbol real del proyecto (se omiten `node_modules`, `target` y `.git`), es la siguiente:

```
.
+-- backend/
|   +-- src/main/java/ec/edu/uteq/pfcbackend/
|   |   +-- client/           OpenLibraryClient, OpenLibraryResult
|   |   +-- config/          SecurityConfig, RedisCacheConfig, WebClientConfig,
|   |   |                   WebServiceConfig, OpenApiConfig, CacheablePage
|   |   +-- controller/      Auth, Libro, Autor, Editorial, Idioma, EstadoLibro,
|   |   |                   DocsRedirect
|   |   +-- dto/             ApiResponse, LibroEnriquecidoResponse, LoginRequest, ...
|   |   +-- entity/          Libro, Autor, Editorial, Idioma, EstadoLibro, Usuario
|   |   +-- exception/       GlobalExceptionHandler, BusinessException, ...
|   |   +-- repository/      LibroRepository, UsuarioRepository, ...
|   |   +-- security/        JwtService, JwtAuthenticationFilter, TokenBlacklistService
|   |   +-- service/         LibroService(+Impl), AutorService(+Impl), ...
|   |   +-- ws/              LibroCatalogoEndpoint (SOAP), LibroNoEncontradoSoapException
|   +-- src/main/resources/
|   |   +-- application.yml
|   |   +-- db/migration/     V1..V8 (Flyway)
|   |   +-- libro-catalogo.xsd
|   +-- src/test/java/ec/edu/uteq/pfcbackend/
|   |   +-- integration/      BaseIntegracionTest, AuthIntegracionTest, LibroIntegracionTest
|   |   +-- client/           OpenLibraryClientTest
|   |   +-- repository/       LibroRepositoryTest
|   |   +-- service/          LibroServiceImplTest, AutorServiceImplTest
|   +-- Dockerfile, pom.xml, mvnw
+-- frontend/src/app/
|   +-- core/                 guards/auth.guard, interceptors, models, services
|   +-- pages/                login/, libros/, libro-form/, libro-detalle/, autores/, autor-form/
|   +-- shared/               notification-banner
+-- nginx/nginx.conf          reverse proxy de entrada (unico puerto publicado)
+-- docker-compose.yml        5 servicios: postgres, redis, app, frontend, nginx
+-- docs/
|   +-- adr/                  ADR-001 a ADR-005
|   +-- arquitectura/         C4, diagrama de clases y ER (puml + png)
|   +-- informe/              informe-unidad-iv.tex, apache-bench.md, auditoria-owasp.md,
|   |                           reflexion-maria.md, reflexion-marlon.md, reflexion-kevin.md,
|   |                           lighthouse/
|   +-- postman/              SGB-API.postman_collection.json (29 requests)
+-- .env.example              JWT_SECRET, COOKIE_SECURE, etc.
+-- README.md
```

Listing 5: Estructura del repositorio.

Anexo C: Análisis y discusión

Las cuatro preguntas que se responden a continuación son las del Anexo C de la guía de práctica (*guia_practica_experimental_IV_app_web.pdf*). Cada nivel de Bloom se indica entre corchetes, como en el enunciado. Las respuestas se basan únicamente en la evidencia de este repositorio (ADRs, benchmark, auditoría OWASP y código real).

Pregunta 1 (Evaluación / Bloom nivel 6)

Revisando el PFC completo (PE-U1 a PE-U4): ¿cuál fue la decisión técnica más difícil que tomó el equipo? ¿Qué alternativas consideraron? ¿Con el conocimiento actual, tomarían la misma decisión o una diferente? Justifique con criterios técnicos referenciados.

La decisión más difícil fue la elección del framework de backend, porque concentraba los mayores costos de migración y porque las tres alternativas que el propio PDF declara válidas (Laravel 11.x, ASP.NET Core 8.x y Spring Boot 3.x) eran técnicamente razonables (ADR-005). Se consideraron las tres: Laravel se descartó por falta de experiencia real del equipo con PHP y por el tipado dinámico (un error de tipo en un DTO con reglas de autorización por rol solo se detectaría en ejecución); ASP.NET Core se descartó pese a ser la opción de mayor rendimiento bruto en TechEmpower Round 23 (609 966 req/s frente a 243 639 req/s de Spring en la prueba Fortunes), porque el rendimiento no era la restricción activa de este proyecto. Se eligió Spring Boot por el dominio previo del equipo con Java, el tipado estático y un ecosistema de testing maduro (Testcontainers, JUnit 5). Con el conocimiento actual el equipo tomaría la misma decisión: el resultado del proyecto (5 recursos CRUD, JWT, cache-aside, SOAP contract-first, 48 tests y un Docker Compose de 5 servicios) no mostró que el rendimiento de Spring fuera una limitación real, y la base de conocimiento previa fue determinante con un plazo acotado. La evidencia de desempeño (P99 de 36 ms con cache frío y 30 ms con cache caliente en la prueba de carga) respalda retrospectivamente que el factor de elección no debía ser el rendimiento bruto.

Pregunta 2 (Síntesis / Bloom nivel 5)

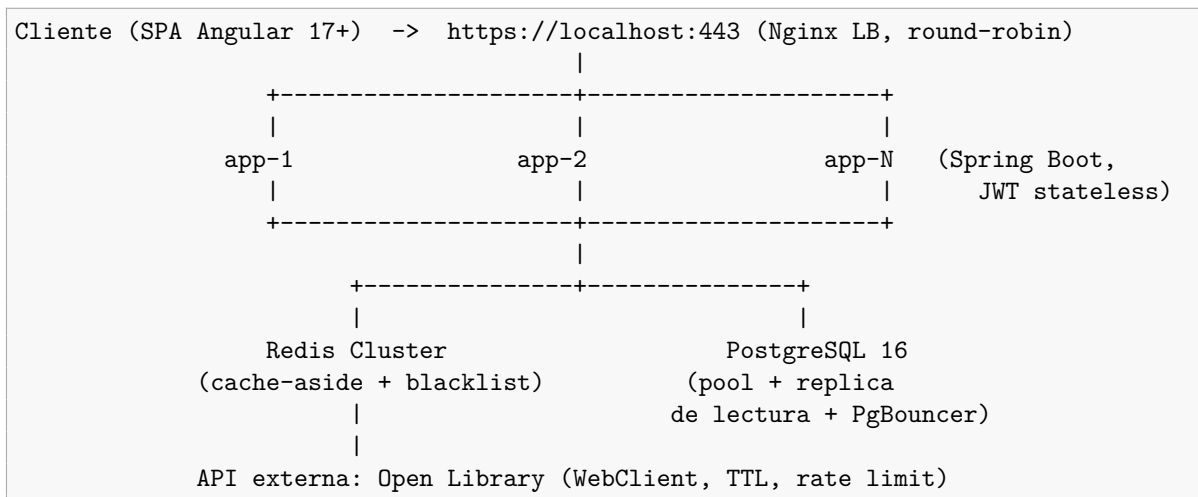
Los resultados de Apache Bench mostraron _____ req/s con _____ % de error. Si se exigiera que el sistema soporte 500 usuarios concurrentes con tasa de error 0 %, ¿qué cambios de arquitectura y código serían necesarios? Dibuje la arquitectura escalada.

Los resultados de Apache Bench mostraron **802.66 req/s** con la cache fría y **986.80 req/s** con la cache caliente, con **0 % de error** en ambas corridas (n=500, c=20, endpoint GET /api/v1/libros contra Nginx). Para soportar 500 usuarios concurrentes con 0 % de error, el cambio de mayor impacto no es de código sino de topología, porque la autenticación JWT stateless ya permite escalado horizontal sin sesiones pegajosas. Los cambios serían:

- **Escalar el plano de aplicación:** varias réplicas de la app Spring Boot detrás de Nginx en modo *load balancer* (upstream con round-robin), en vez de una sola instancia. El JWT HttpOnly (ADR-003, dual cookie + header Authorization) funciona igual en cualquier réplica porque el servidor no guarda estado de sesión.

- **Alta disponibilidad de Redis:** hoy la cache-aside y la blacklist de `jti` comparten una sola instancia de Redis, reconocido en el ADR-003 como un punto único de fallo. Para 500 concurrentes se requeriría Redis Sentinel o Cluster, y separar la blacklist de la cache en instancias distintas.
- **Escalar PostgreSQL:** aumentar el pool de conexiones (`HikariCP`), usar un *connection pooler* intermedio (`PgBouncer`) y añadir una réplica de lectura; las escrituras son infrecuentes frente a las lecturas del catálogo.
- **Mitigar el *cache stampede*:** el ADR-003 documenta que hoy el riesgo existe y no está mitigado (TTL que expira con muchas lecturas simultáneas y `allEntries=true` en cada escritura). A 500 usuarios concurrentes se justificaría `@Cacheable(sync=true)` para serializar la reconstrucción de una key.
- **Rate limiting hacia la API externa:** el consumo de Open Library pasa por el mismo `WebClient` (sección de desarrollo); a esa concurrencia se requeriría un límite de peticiones por ventana de tiempo y un TTL de cache acorde al dominio (los 5 minutos actuales del cache-aside están dentro del rango que la guía sugiere para datos que cambian poco).

Arquitectura escalada propuesta:



Pregunta 3 (Análisis / Bloom nivel 4)

Analice los 6 principios REST de Fielding contra la API REST del PFC: ¿cuáles cumple completamente, cuáles parcialmente y cuáles no? Para los principios no cumplidos: ¿cuál es el costo de no cumplirlos en este contexto específico?

Tabla 5: Cumplimiento de los 6 principios de Fielding en la API del PFC.

Principio	Cumplimiento	Evidencia en el repo
Client-Server	Completo	Angular 17+ consume la API; el servidor no renderiza la vista
Stateless	Completo	JWT firmado en cada petición; blacklist de <code>jti</code> en Redis (ADR-003)
Cacheable	Parcial	Cache-aside en Redis (TTL 5 min); sin cabecera <code>Cache-Control</code> hacia el cliente
Uniform Interface	Parcial	URIs semánticas (<code>/api/v1/libros</code>) y verbos HTTP; sin HATEOAS/hipermedia
Layered System	Completo	Nginx como reverse proxy delante de la app; el cliente solo ve Nginx
Code-On-Demand	No	No se envían scripts descargables; es un principio opcional en REST

Cacheable (parcial): las respuestas se cachean en el servidor (Redis), pero la API no emite la cabecera `Cache-Control`, por lo que el navegador o un CDN no pueden reutilizar la respuesta. El costo en este contexto es bajo: el consumidor real es una SPA del propio equipo y el cache-aside ya reduce la carga de PostgreSQL; un CDN solo aportaría si hubiera usuarios externos dispersos.

Uniform Interface (parcial): faltan los *links* de hipermedia (HATEOAS); las respuestas JSON no enlazan los recursos relacionados. El costo es moderado: el cliente conoce las URIs de forma estática (convención del equipo), por lo que no se pierde adaptabilidad, pero un consumidor tercero tendría que leer la documentación OpenAPI en vez de descubrir los recursos en la propia respuesta. Para un PFC con consumidores conocidos es aceptable.

Code-On-Demand (no cumplido): REST lo define como opcional y el equipo no lo usa (la SPA transfiere el código en build, no por HTTP dinámico). El costo es nulo: ningún escenario del PFC requiere ejecutar código provisto por el servidor en tiempo de ejecución.

Pregunta 4 (Evaluación / Bloom nivel 6)

Comparando el PFC con un sistema web de producción real del mismo dominio (buscar un caso de estudio publicado), ¿qué características de calidad (ISO/IEC 25010:2023) están cubiertas por el PFC y cuáles requerirían trabajo adicional para ser comercialmente viables? ¿Cuánto tiempo estimaría para hacerlo comercialmente viable?

Tomando el catálogo de una biblioteca como dominio, las características de ISO/IEC 25010:2023 que el PFC cubre con evidencia son: **adecuación funcional** (5 recursos CRUD, autenticación con roles, SOAP y REST), **seguridad** parcial (auditoría OWASP A01–A07 con 6 hallazgos mitigados y 2 brechas documentadas), **eficiencia de desempeño** en carga moderada (P99 30–36 ms a 20 concurrentes) y **mantenibilidad** (arquitectura en capas, ADRs, 48 tests y 60 % de cobertura de instrucciones medida por JaCoCo en el último build).

Requerirían trabajo adicional para ser comercialmente viable: **fiabilidad** (sin monitoreo, *health checks* ni respaldo/restauración de datos documentados), **usabilidad** (sin auditoría de accesibilidad WCAG ni pruebas con usuarios reales), **portabilidad** parcial (Docker Compose sí, pero sin *secrets* en el repositorio ni *pipeline* CI/CD automatizado) y **seguridad** restante (las dos brechas de la auditoría y *rate limiting* frente a consumo abusivo).

Como caso de estudio publicado del mismo dominio, Koha ([Breeding, 2017](#)) es el primer sistema integrado de gestión bibliotecaria (ILS) de código abierto, en producción ininterrumpida desde el año 2000 (creado originalmente para el Horowhenua Library Trust en Nueva Zelanda) y usado hoy en bibliotecas de todo tamaño, incluyendo consorcios completos — una escala de producción real de 25 años que este PFC no alcanza ni pretende alcanzar en un semestre. La comparación en tres características de ISO/IEC 25010:2023 deja ver la brecha real, no solo la esperada:

- **Adecuación funcional:** Koha cubre módulos de circulación, catalogación bajo estándares bibliotecarios reales (MARC 21, UNIMARC, Z39.50/SRU), adquisiciones, publicaciones seriadas y un OPAC público, además de protocolos de integración con hardware externo (SIP2/NCIP para autopréstamo y RFID). Este PFC cubre 5 recursos CRUD genéricos y dos protocolos de exposición (REST y SOAP), sin ningún estándar bibliotecario real, porque no era el objetivo del curso.
- **Fiabilidad:** Koha tiene 25 años de operación en producción real, con un proceso de release formal respaldado por una comunidad activa. Este PFC no tiene monitoreo más allá de los healthchecks de Docker, ni una estrategia de respaldo/restauración de PostgreSQL documentada.
- **Portabilidad/Interoperabilidad:** Koha se integra con sistemas de terceros mediante protocolos bibliotecarios estándar (SIP2, Z39.50/SRU para catalogación compartida entre bibliotecas). Este PFC integra un único servicio externo (Open Library, vía REST) y no implementa ningún estándar de interoperabilidad bibliotecaria.

Desglose temporal estimado para acercar este PFC a un nivel de madurez comparable, más granular que el rango de 2 a 3 meses ya mencionado: 1 semana para monitoreo básico (métricas y logs centralizados), 1 semana para respaldos automáticos de PostgreSQL, 2 semanas para cerrar las 2 brechas de la auditoría OWASP (*rate limiting* y actualización de *jwt*), 2 semanas para pruebas de accesibilidad WCAG y ajustes de UI, y 2 semanas para un pipeline CI/CD con despliegue de múltiples réplicas y TLS gestionado (8 semanas en total, con una persona dedicada). Ninguna de esas 8 semanas cierra la brecha de interoperabilidad bibliotecaria real (MARC, Z39.50, SIP2) que Koha sí tiene: implementar esos estándares es un proyecto de dominio en sí mismo, no una tarea de endurecimiento del sistema actual, y por eso se mantiene fuera de la estimación de viabilidad comercial de este PFC en su forma actual.